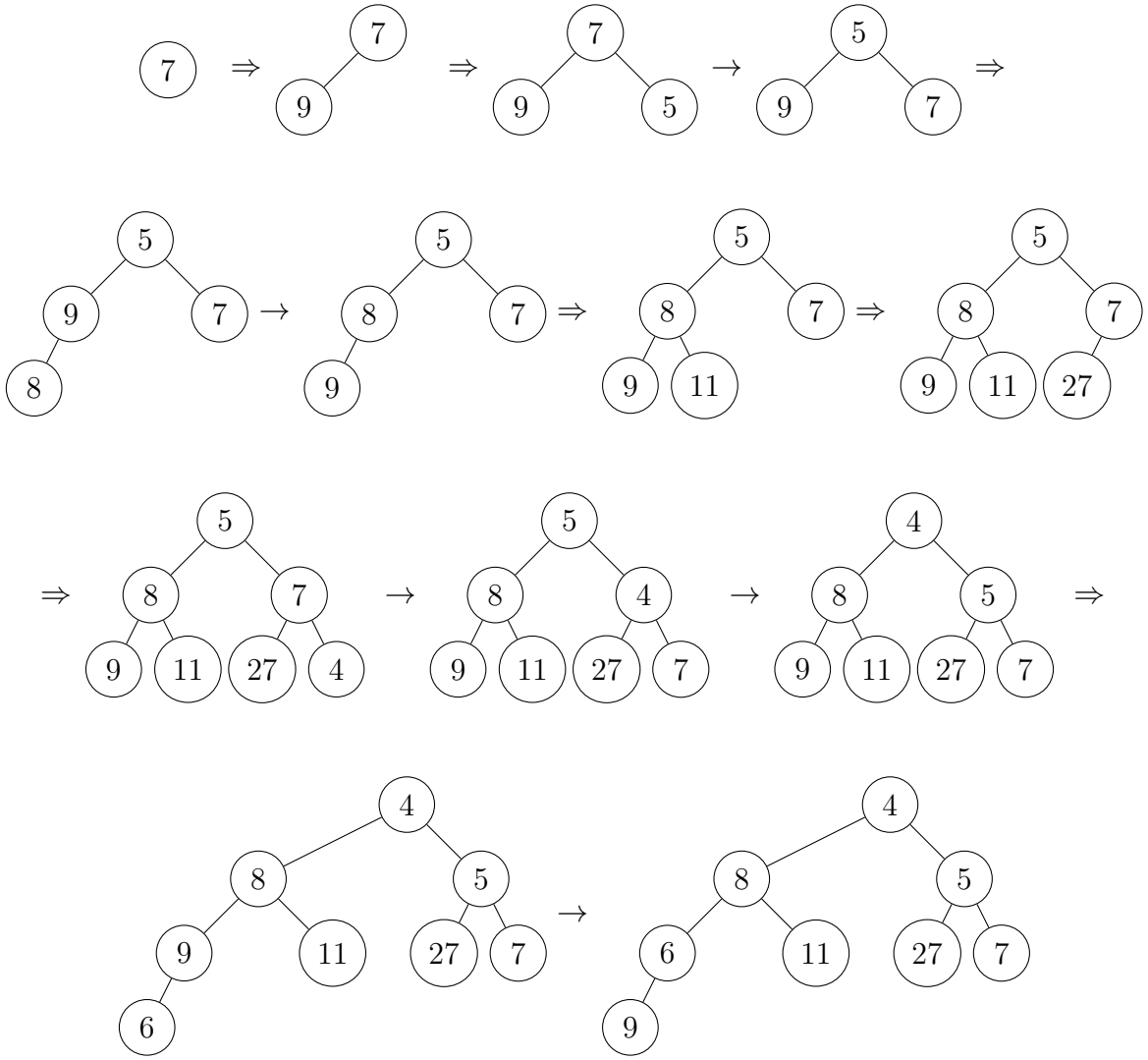
DS Written Homework 5

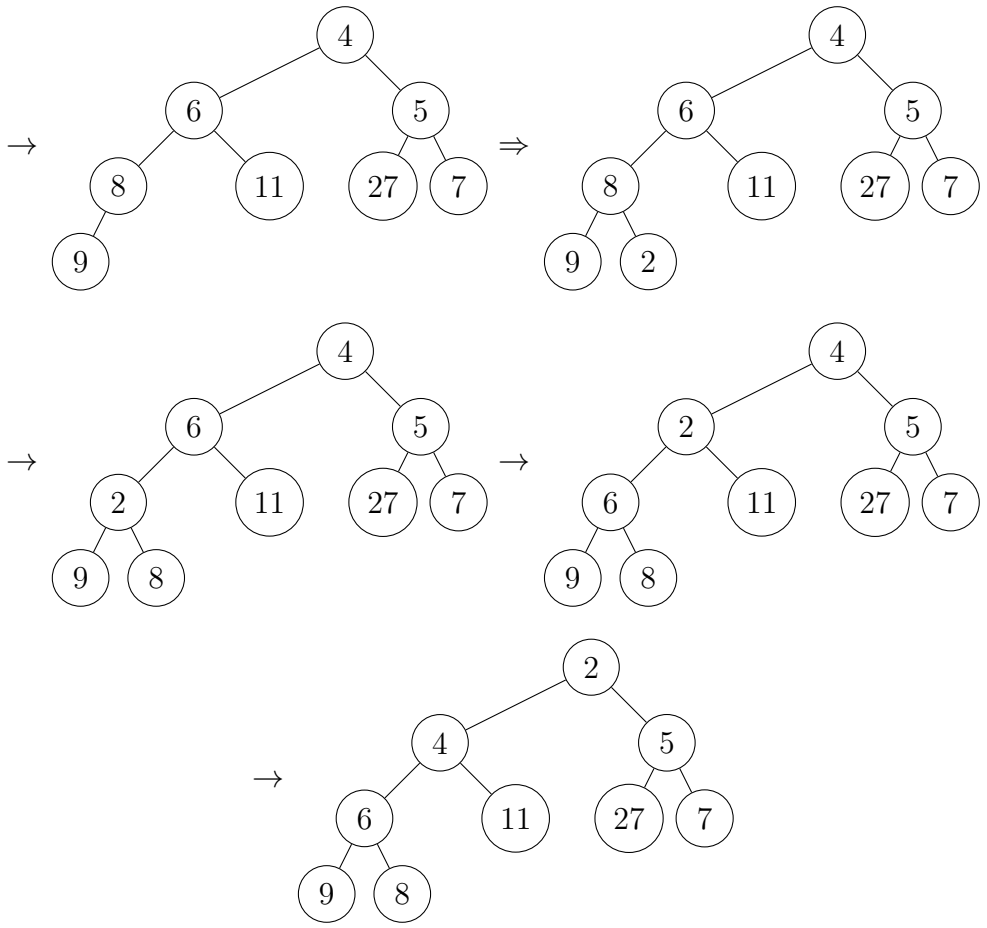
Student ID: b10201054

May 29, 2025

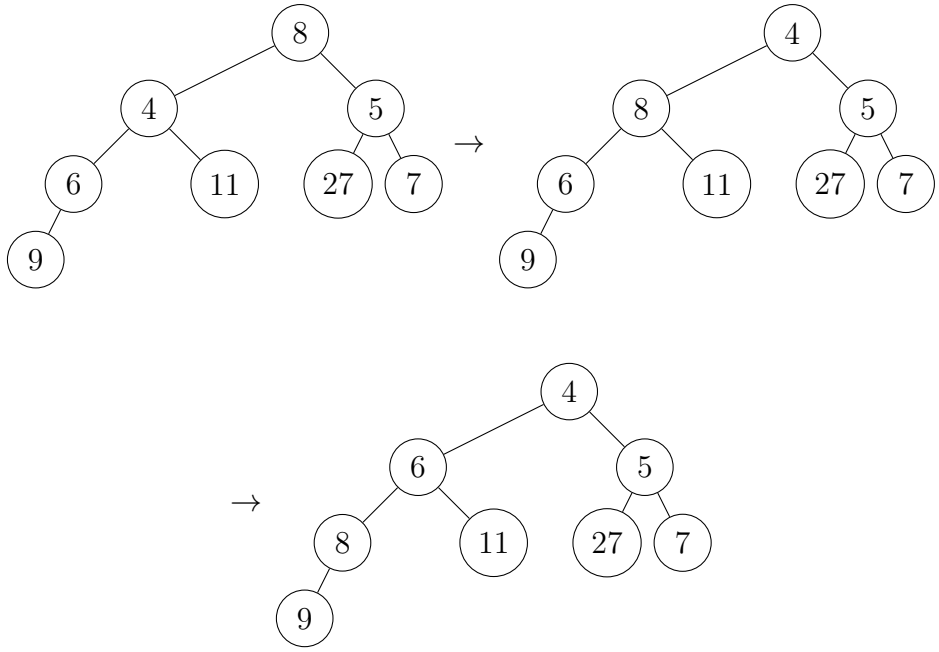
1 Binary Heap Operation

(a) $\Rightarrow$ means inserting next node, $\rightarrow$ means swimming.





(b) ⇒ means inserting next node, → means swimming.



2 Binomial Heap

(a) If there are two same degree binomial trees, we will merge them into a degree + 1 binomial tree. If binomial heap contains 10 bionomial trees, their degree must be different, so if we want the smallest number of nodes, we can choose smallest 10 numbers of the degree. Hence, the smallest possible number of nodes is $\sum_{k=0}^{9} 2^k = 2^10 - 1 = 1023$.

(b) Insert a node has $O(1)$ complexity.

Merge two trees has $O(1)$ complexity.

Insert m node has m insertion and at most $\log_2 m$ merge.(Without considering the trees already in heap.)

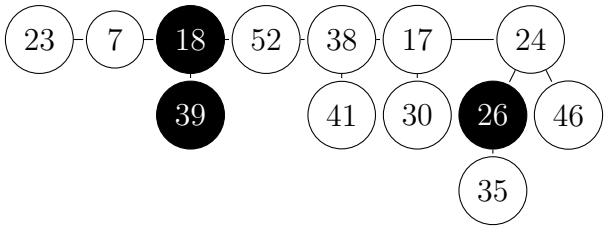
The worst case of inserting time complexity happens when we need to merge all the binomial trees in the current binomial heap. By (a) we can find the largest possible number of an n nodes binomial heap is $N = \log_2 n$.

Hence, the time complexity of inserting m node is $O(m + \log_2 m + \log_2 n) = O(m + \log n)$.

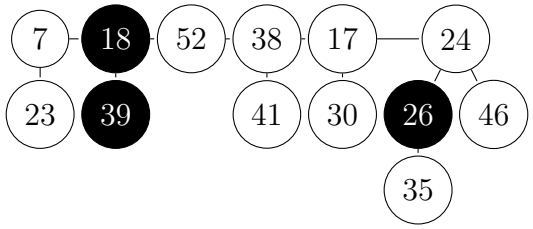
3 Fibonacci Heap

(a) Result:

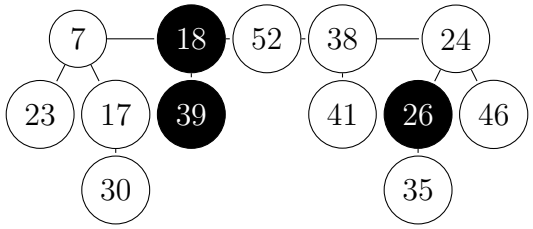
Step 1. Extract Min = 3:



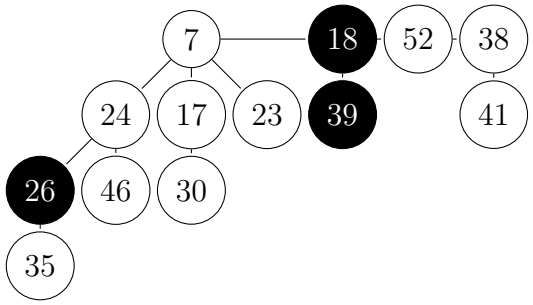
Step 2. From the min = 17, record the root degree, find the same degree tree then merge them. In this case we record [(17, 1), (24, 2), (23, 0)], then the next node (7,0) is same degree as (23, 0), merge:



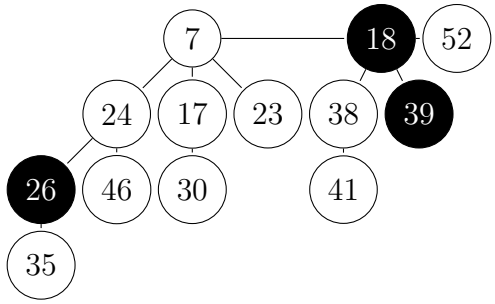
Step 3. After merged, the list we recorded is [(17, 1), (24, 2)], find (7, 1) is same degree as (17, 1), merge:



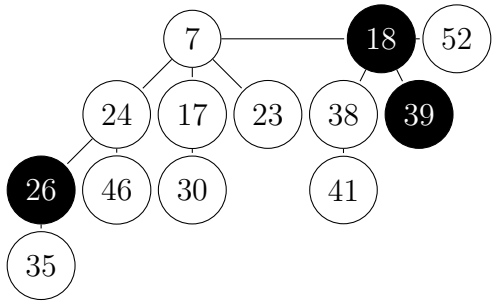
Step 4. After merged, the list we recorded is [(24, 2)], find (7, 2) is same degree as (24, 2), merge:



Step 5. After merged, the list we recorded is [(7, 3)], keep moving forward and record. The list we recorded is [(7, 3), (18, 1), (52, 0)], the next node (38, 1) has same degree as (18, 1), merge:

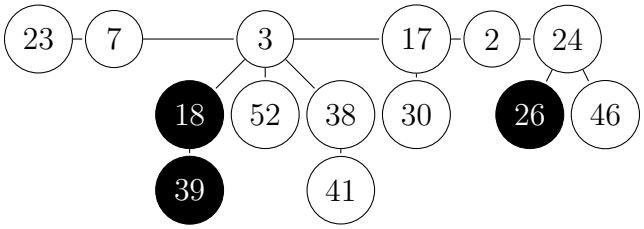


Step 6. After merged, the list we recorded is [(7, 3), (18, 2)], keep moving forward and record. The list we recorded is [(7, 3), (18, 2), (52, 0)], the next node is (7, 3), hence the process is complete.

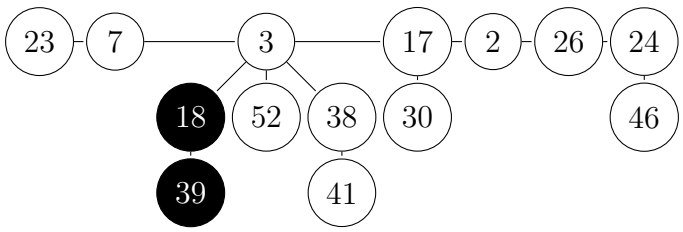


(b) Result:

Step 1. Decrease 35 to 2, then move 2 to root list:



Step 2. 26 lost a kid, since 26 had already lost a kid, move 26 to root list and reset the lost kid number:



Step 3. Change the min number to 2.

4 Modified Fibonacci Heap

(a) In the modified Fibonacci heap, the node will be cascading cut only if the mark of the node reach two. That is, the node had lost its child two times, and now it lost its child again. Hence, the potential move of the modified Fibonacci heap is $\phi(H) = t(H) + 2 \times m_2(H)$, where $m_2(H)$ means the number of nodes which has mark = 2.

i. Insertion won't change any of the $m_2(H)$, and $t(H)$ will increase by 1, so the cost of insertion is $\phi(H') - \phi(H) = 1$. Amortized cost: $O(1)$.

ii. When we decrease key k, we will execute cut 1 time, then start cascading-cut. In the cascading-cut, we only execute cut when we find the node with mark = 2. Hence, cut will be executed $c \leq m_2(H)$ times when doing cascading-cut. For each cut executed, the tree number will increase by 1. Then the potential function

$$\phi(H') \leq (t(H) + c + 1) + 2 \times (m_2(H) - c + 1) = \phi(H) + 2 - c$$

Hence, the amortized cost is $O(1)$.

iii. Delete-min won't change any of the $m_2(H)$, but the tree number will increase the children number of min node, so $\phi(H') - \phi(H) = D(n)$, which means the amortized cost is $O(D(n))$

iv. Union only change the minimum number of the heap, so that $\phi(H') - \phi(H) = 0$. Hence, the amortized cost is $O(1)$.

(b) The minimum number of nodes can be recognized as the maximum number of nodes can be deleted from a binary tree without breaking the structure. For the degree = 11 tree, the node number is $\sum_{i=0}^{10}(1+\sum_{j=0}^{i}(2^j))$. Since we can mark a node twice, we can delete the largest and second branches. That is, the number is $1+1+\sum_{i=2}^{10}(\sum_{j=0}^{i-2}(1+\sum_{k=0}^{j}(2^k)))$.

Observe the recursion, we can find that the number of nodes are in the sequence 1, 1, 1, 2, 3, 5,... which is also a Fibonacci sequence. Hence the degree 11 tree is the Fibonacci number $F_{12}+1 = 145$.

5 Disjoint Set

(a) Result:

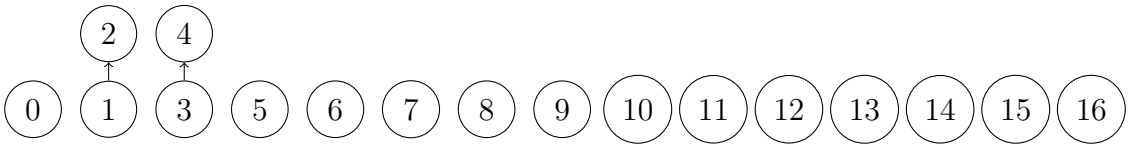


Figure 1: union(1, 2), union(3, 4)

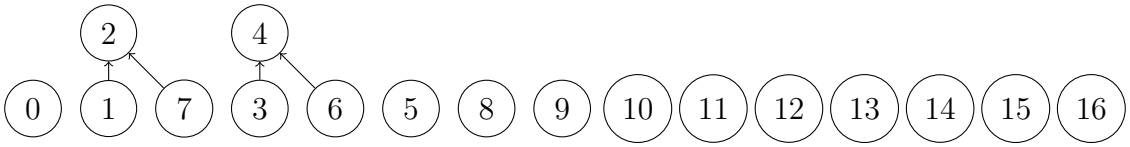


Figure 2: union(1, 7), union(3, 6)

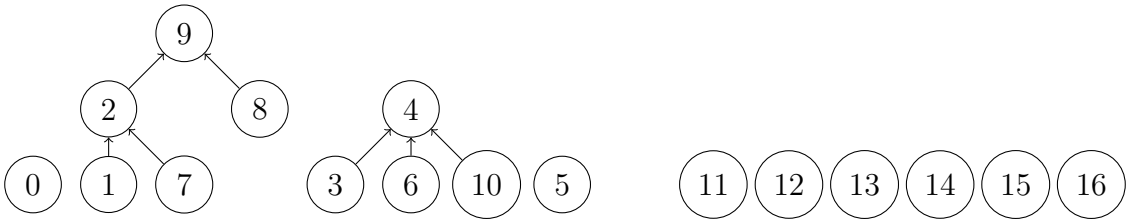


Figure 3: union(8, 9) then union(1, 8), union(3, 10)

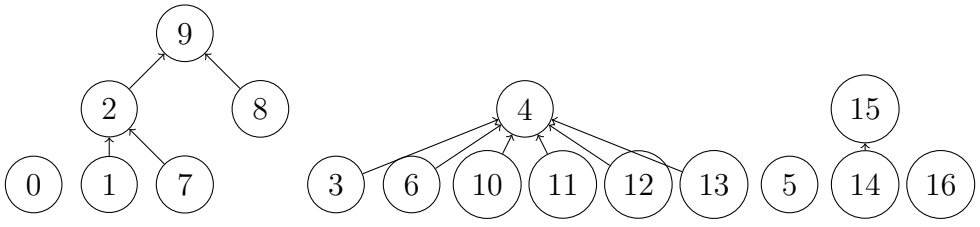


Figure 4: union(3, 11), union(3, 12), union(3, 13), union(14, 15)

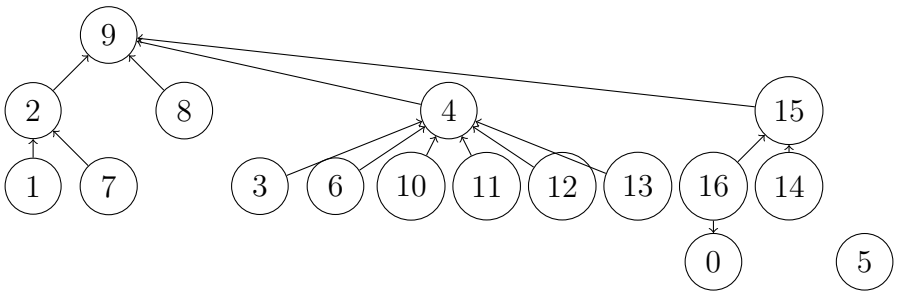


Figure 5: union(16, 0), union(14, 16), union(1, 3), union(1, 14)

(b) Result:

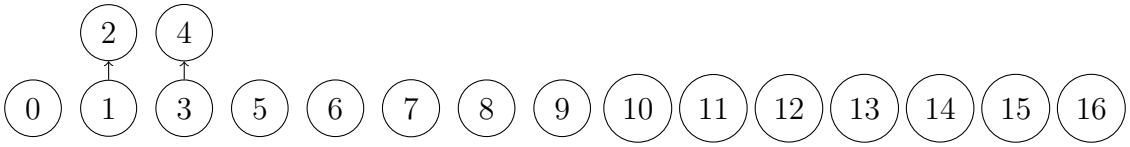


Figure 6: union(1, 2), union(3, 4)

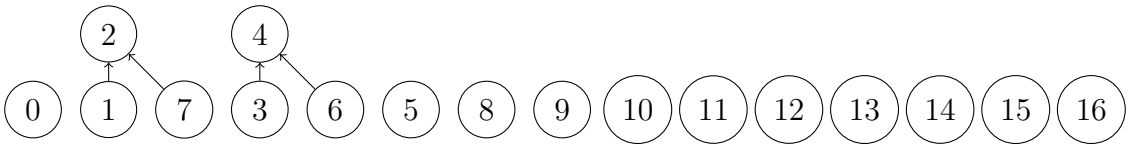


Figure 7: union(1, 7), union(3, 6)

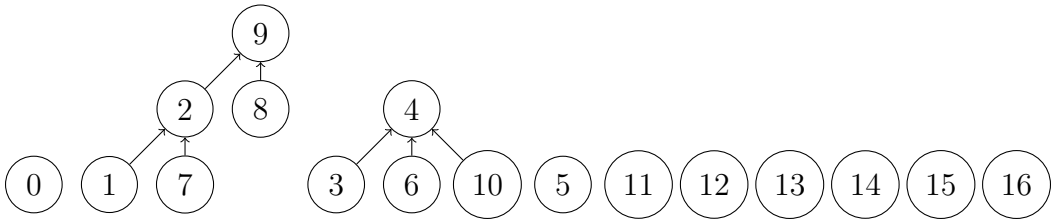


Figure 8: union(8, 9) then union(1, 8), union(3, 10)

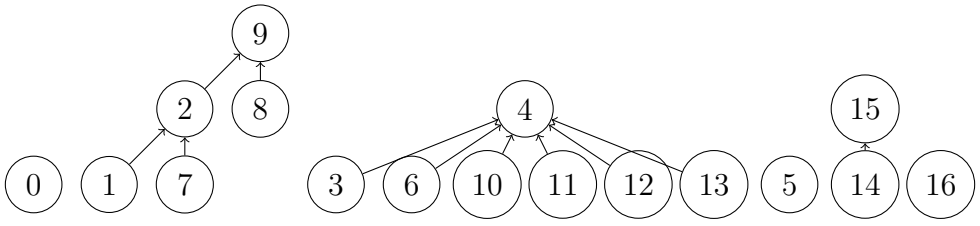


Figure 9: union(3, 11), union(3, 12), union(3, 13), union(14, 15)

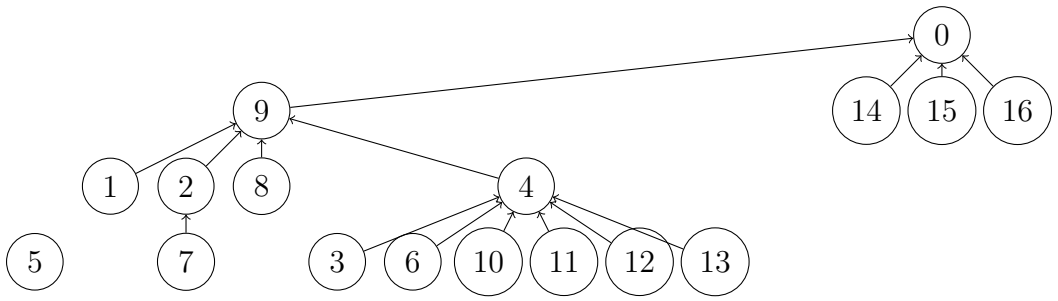


Figure 10: union(16, 0), union(14, 16), union(1, 3), union(1, 14)

6 Disjoint Set Algorithm

Add another attribute link, and change the functions MAKE-SET, LINK, PRINT-SET. Then we can reach the goal we want.

```
   Input: x
   Output: print the set of x
 1 Function MAKE-SET(x):
 2 |   x.parent ⟵ x;
 3 |   x.rank ⟵ 0;
 4 |   x.link ⟵ x;
 5 End Function
 6 Function LINK(x, y):
 7 |   temp = y.link;
 8 |   y.link = x.link;
 9 |   x.link = temp;
10 |   if x.rank > y.rank then
11 |   |   y.parent ⟵ x;
12 |   else
13 |   |   x.parent ⟵ y;
14 End Function
15 Function PRINT-SET(x):
16 |   root = x;
17 |   while x.link ≠ root do
18 |   |   print(x);
19 |   |   x ⟵ x.link;
20 |   end
21 |   print(x);
```

Algorithm 1: Print-Set(x)